Training GradientBoostingRegressor

We follow each step one by one written below :

Input: training set $\{(x_i, y_i)\}_{i=1}^{n}$, a differentiable loss function $L(y, F(x))$, number of iteration.

1. Initialize $f_0(x) = \arg\min_\gamma \sum_{i=1}^{N} L(y_i, \gamma)$.
2. For $m = 1$ to M:
   (a) For $i = 1, 2, \ldots, N$ compute
   $$r_{im} = -\left[\frac{\partial L(y_i, f(x_i))}{\partial f(x_i)}\right]_{f=f_{m-1}}.$$
   (b) Fit a regression tree to the targets r_{im} giving terminal regions $R_{jm},\ j = 1, 2, \ldots, J_m$.
   (c) For $j = 1, 2, \ldots, J_m$ compute
   $$\gamma_{jm} = \arg\min_{\gamma} \sum_{x_i \in R_{jm}} L\left(y_i, f_{m-1}(x_i) + \gamma\right).$$
   (d) Update $f_m(x) = f_{m-1}(x) + \sum_{j=1}^{J_m} \gamma_{jm} I(x \in R_{jm})$.
3. Output $\hat{f}(x) = f_M(x)$.

From the next page, I explained what each line denotes in my notebook. The most important part here is to choose the Loss Function that we want to use in Gradient Boosting.

training set → $\{(x_i, y_i)\}_{i=1}^{n}$. (n = number of rows)

x		y
cg	iq	salary
3.8	50	75 → (x_1, y_1)
8.9	100	50 → (x_2, y_2)
5.3	200	30 → (x_3, y_3)

A differentiable loss Function $L(y, F(x))$:

→ We need a Loss function (a function which we use to calculate to quantify the error of a model, E.g., $MSE = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$) which is differentiable.

→ $F(x)$ is the function we use to predict the output column, $\hat{y}$. so, $F(x) = \hat{y}$ and $L(y, \hat{y})$.

M:

Number of models/estimators we want to use in Gradient Boosting.

CS Scanned with CamScanner

What is our objective in Gradient Boosting?

As we already know, in Gradient Boosting, the 1st model returns the mean of the output column as prediction and the rest of the models return/predict the error of its previous model's error.

$\therefore \hat{y}$ after all the model are trained;

$$\hat{y} = b_0(x) + b_1(x) + b_2(x) + \cdots + b_m(x)$$

$b_0(x)$: Base Model — 1st model which returns mean as its prediction. Also called 'Base Model'.

$b_1(x) + b_2(x) + \cdots + b_m(x)$: Tree Models — The rest models predicted error. 1st tree model = $b_1(x)$.

Now assume we don't know if the 1st model, $b_0(x)$ returns mean or not, rest models error or not. Now we'll find their equations from the Loss function we'll take.

Scanned with CamScanner

In our case we'll take MSE as Loss function:

$$L = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

For mathmetical convenience, we'll take $n=2$ which doesn't have any effect on what Loss function stands for.

$$\therefore L = \frac{1}{2}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

Step 1 Initialize $f_0(x) = \arg\min_\gamma \sum_{i=1}^{N} L(y_i, \gamma)$

N = total rows

Here, $f_0(x) = \arg\min_\gamma \sum_{i=1}^{n} L(y_i, \gamma)$

↳ Instead of y_i $\gamma = \hat{y}_i$

$$\therefore f_0(x) = \arg\min_\gamma \sum_{i=1}^{n} \frac{1}{2}(y_i - \gamma)^2$$

$L(y_i, \gamma)$

This equation of $f_0(x)$ telling to find such gamma (γ) for what this equation will be minimized i.e. $\frac{d}{d\gamma}(\text{equation}) = 0$ and you'll find the minimum of $\gamma(\hat{y})$.

Scanned with CamScanner

So,

$$b_0(x) = \frac{1}{2}\sum_{i=1}^{n}(y_i - \gamma)^2$$, but find such γ for what $b_0(x)$ will be minimized..

i.e. $\frac{d}{d\gamma} b_0(x) = 0$

$$\therefore \frac{d}{d\gamma}\{b_0(x)\} = 0$$

$$\Rightarrow \frac{d}{d\gamma}\left\{\frac{1}{2}\sum_{i=1}^{n}(y_i - \gamma)^2\right\} = 0$$

$$\Rightarrow \frac{1}{2}\cdot\sum_{i=1}^{n}\cdot\frac{d}{d\gamma}(y_i - \gamma)^2 = 0$$

$$\Rightarrow \frac{1}{2}\cdot\sum_{i=1}^{n}\cdot 2(y_i - \gamma)\cdot\frac{d}{d\gamma}(y_i - \gamma) = 0$$

$$\Rightarrow \sum_{i=1}^{n}(y_i - \gamma)\cdot(-1) = 0$$

$$\Rightarrow \sum_{i=1}^{n}(\gamma - y_i) = 0$$

$$\boxed{\frac{d}{d\gamma}\{b_0(x)\} = \sum_{i=1}^{n}(\gamma - y_i) = 0}$$

For the dataset:

	x	y
	cg	iq	salary
	3.8	50	75
	8.9	100	50
	5.3	200	30

$$\Rightarrow \sum_{i=1}^{3} (Y - \hat{Y}_i) = 0$$

$$\Rightarrow (Y - 75) + (Y - 50) + (Y - 30) = 0$$

~~$3Y - 155 = 0$~~

$$\Rightarrow 3Y - 75 - 50 - 30 = 0$$

$$\Rightarrow Y = \frac{75 + 50 + 30}{3 \text{ (row numbers)}}$$

$\therefore Y = \hat{Y}$ = mean of Y column.

$\therefore f_0(x)$ = mean of output column

We've already seen that the 1st model quantifies nothing but the mean of output column as its ~~Blood cell~~ prediction and we've seen the proof.

But if we've used any other Loss Function, then we would get something else instead of mean ~~maybe~~.

Step 2: for m=1 to M (range(1, M+1))

In step 2, there are 4 steps which is for every Decision Tree model ($b_1(x), b_2(x)$ ---) since the loop runs for each Tree model. Whatever happens for one model, happens for all of them. Lets see what happens for one Tree model, m=1:

(a) For i (row) = 1, 2, ... N, compute:

$$r_{im} = -\left[\frac{\partial L(y_i, b(x_i))}{\partial b(x_i)}\right]_{b=b_{m-1}}$$

→ r means residual / pseudo-residual error (Actual value - predicted value).

→ So for each model, we traverse each row to calculate the residual error => r_{im}.

For m=1:

$$r_{i1} = -\left[\frac{\partial L(y_i, b(x_i))}{\partial b(x_i)}\right]_{b=b_0}$$

CS Scanned with CamScanner

$$\Rightarrow r_{i1} = -\left[\frac{\partial}{\partial \hat{y}_i} L(y_i, \hat{y}_i)\right]_{f=f_0}$$

From 1st Page:
$\hat{y} = f(x)$
$\Rightarrow \hat{y}_i = f_i(x)$

Here, r_{i1} = Residual error for any row for the 1st Tree model. (any row = a single row)

$$L(y, \hat{y}) = L = \frac{1}{2}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2 \text{ for all rows.}$$

$$L(y_i, \hat{y}_i) = L = \frac{1}{2}(y_i - \hat{y}_i)^2 \text{ for single row.}$$

$$\Rightarrow r_{i1} = -\left[\frac{\partial}{\partial \hat{y}_i}\left\{\frac{1}{2}(y_i - \hat{y}_i)^2\right\}\right]_{f=f_0}$$

$$\Rightarrow r_{i1} = -\left[\frac{1}{2} \cdot 2(y_i - \hat{y}_i) \cdot (-1)\right]$$

$$\Rightarrow r_{i1} = \left[(y_i - \hat{y}_i)\right]_{f=f_0}$$

$$\Rightarrow r_{i1} = \left[(y_i - f(x_i))\right]_{f=f_0} \quad \left| \begin{array}{l} \hat{y} = f(x) \\ \hat{y}_i = f(x_i) \end{array} \right.$$

$$\therefore r_{i1} = \left[(y_i - f_0(x_i))\right]$$

$y.mean()$ which first / base model returned.

Scanned with CamScanner

$\therefore r_{i1} = y_i - f_0(x_i)$

$\Rightarrow r_1 = y - f_0(x)$, residual error for the

first Tree / second model is the difference

between Actual Output column and predicted output column of the base/first model.

not a Tree model

x		y	1st/base model	2nd model
iq	cg	salary	$f_0(x)$	r_1
20	3.7	53	42	11
10	3.3	33	42	-9
33	3.0	40	42	-2

(b) Fit a regression tree to the targets r_{im} giving terminal regions R_{jm}, $j = 1, 2, \ldots J_m$.

Fit a regression tree to the targets r_{im}!

target refers to the output column that

will be given in regression tree to train. In this case, it is said that the target column is r1 which we just calculated, not salary.

```
tree = DecisionTreeRegressor(max_depth = 1)
tree.fit(X, r1)
```

For simplicity purpose.

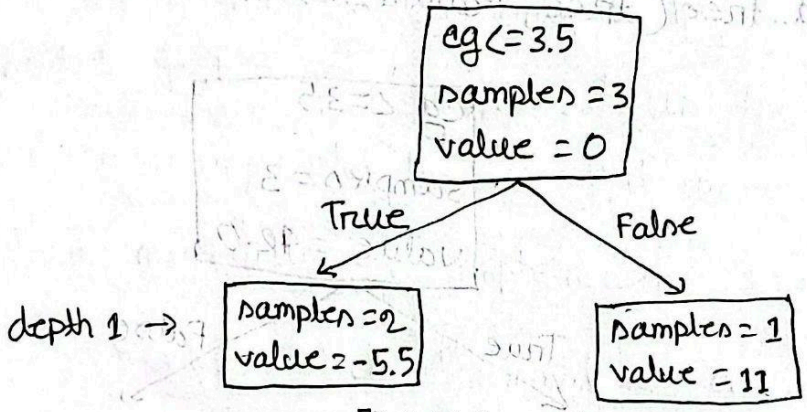


Fig: Using plot_tree()

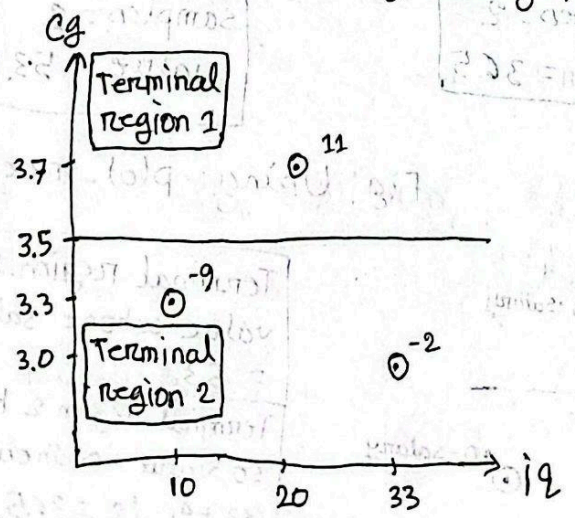


Fig: plot_decision_regions(...)

What value means ? r1 value.
Terminal region 1	Right sub Tree, has 1 value whose r1 = value = 11,
Terminal region 2	Left sub Tree, has 2 values, so value in r2 mean = (-9-2)/2 = -5.5
The reason value - represents r2 value because we trained tree with output array, r1

Scanned with CamScanner

... giving terminal regions $R_{jm}, j = 1, 2, \ldots J_m$;

since 'tree' has only 1 depth, we've only 1 split on cg = 3.5 since splitting condition is 'cg <=3.5'. We've 1 split, so we've 2 terminal regions which we can see in above figure.

j represents terminal-region-number. Here j=1,2.

R " Regression Tree.

Step 2 is a loop for each Tree model and we're quantifying for only one model, m=1.

$\therefore R_{jm} = R_{11}$ represents "Terminal region 1" for 1st Tree model.

$R_{jm} = R_{21}$ represents "Terminal region 2" for 1st Tree model.

(b) Fit a regression tree to the target ----

It said to ~~code~~ only train the tree model ~~boot~~ after what we'll have terminal regions.

Number of leaf nodes = Number of terminal regions.

CS Scanned with CamScanner

We've 3 input rows and 2 leaf nodes / terminal regions. -5.5 goes to 2 rows and 11 to 1 row.

cg	rl	Predicted_rl
3.7	11	11
3.3	-9	-5.5
3.0	-2	-5.5

cg<=3.5 ? -5.5 : 11

C++ ternary.

But we're said in (b) only to train the tree model. But I've also showed us the prediction values using plot_tree(). But here we've to learn to calculate them manually i.e. implementing predict() manually in (c) for each terminal region.

(c) For $j = 1, 2, \ldots J_m$, compute

$$\gamma_{jm} = \underset{\gamma}{\operatorname{argmin}} \sum_{x_i \in R_{jm}} L(y_i, f_{m-1}(x_i) + \gamma)$$

Actually, $\gamma_{jm} = \sum_{x_i \in R_{jm}}^{k} L(y_i, f_{m-1}(x_i) + \gamma)$ and we've to find such γ for what γ_{jm} value will be minimized. k = number of points / rows in the terminal region / leaf node.

CS Scanned with CamScanner

→ x represents input columns

→ γ_{jm} is for

→ γ_{jm} represents the To Be Predicted Value (residual error of the previous model) for each Terminal Region / Leaf Node for the current model ($m=1$).

→ x_i represents the ~~for~~ input rows (eg, i4) belong. (ϵ) to ~~the~~ each Terminal Region of the Regression Tree. That's what $x_i \in R_{jm}$ is.

→ Loss function, $L(y_i, \gamma) = \frac{1}{2} \sum_{i=1}^{n} (y_i - \gamma)^2$ and in this function we're calculating loss i.e. $(y_i - \gamma)^2$ for all the input rows but for γ_{jm} we won't calculate loss for each row and for the rows we need to calculate loss is handled by $\sum_{x_i \in R_{jm}}^{k}$. so in our case

we don't need $\sum_{i=1}^{n}$ because it is already handled by $\sum_{x_i \in R_{jm}}^{k}$

∴ Since m=1,

$$\gamma_{jm} = \sum_{x_i \in R_{jm}}^{k} \frac{1}{2} \cdot \left(y_i - \{b_0(x_i) + \gamma\}\right)^2$$

$$\therefore \gamma_{jm} = \frac{1}{2} \sum \cdot (y_i - b_0(x_i) - \gamma)^2$$

Now, $\frac{d}{d\gamma}(\gamma_{jm}) = 0$ [Find such γ for what γ_{jm} function is minimized]

$$\Rightarrow \frac{d}{d\gamma} \left\{ \frac{1}{2} \sum \cdot (y_i - b_0(x_i) - \gamma)^2 \right\} = 0$$

$$\Rightarrow \frac{1}{2} \cdot \sum \cdot 2(y_i - b_0(x_i) - \gamma) \cdot \frac{d}{d\gamma}(y_i - b_0(x_i) - \gamma) = 0$$

$$\Rightarrow \sum (y_i - b_0(x_i) - \gamma) \cdot (0 - 0 - 1) = 0$$

$$\Rightarrow \sum_{x_i \in R_{jm}}^{k} (y_i - b_0(x_i) - \gamma) = 0$$

$\frac{d}{d\gamma} b_0(x_i)$
$= \frac{d}{d\gamma} \sum_{i=1}^{n} (\gamma - y_i)^1$
$= \sum_{i=1}^{n} 0 \cdot (\gamma - y_i) \cdot 1$
$= 0$

We've to find γ and γ in different for each terminal region since each one has different number of rows / samples.

$j=[1,2]$. So $\gamma_{jm} = \gamma_{11}, \gamma_{21}$. And $m=1$.

For γ_{11}: (Terminal Region 1)

$k=1$ = number of rows in TR1

$$\sum_{x_i \in R_{jm}}^{k} (y_i - b_0(x_i) - \gamma) = 0$$

$$\Rightarrow \sum_{i=1}^{k=1} (y_i - b_0(x_i) - \gamma) = 0$$

$$\Rightarrow 53 - 42 - \gamma = 0$$

$$\therefore \gamma = 11$$ (verified from plot_tree() graph)

For γ_{21}: (Terminal Region 2, has 2 rows, so $k=2$)

$$\sum_{x_i \in R_{jm}}^{k} (y_i - b_0(x_i) - \gamma) = 0$$

$$\Rightarrow \sum_{i=1}^{k=2} (y_i - b_0(x_i) - \gamma) = 0$$

$$\Rightarrow (33 - 42 - \gamma) + (40 - 42 - \gamma) = 0$$

$$\Rightarrow -11 - 2\gamma = 0$$

$$\therefore \gamma = -5.5$$ (verified from plot_tree() graph)

Scanned with CamScanner

To check how many rows / samples each terminal region / Leaf Node has, we eventually needed the plot_tree() graph but we learnt to calculate 'value' manually.

$$(d)\ \text{Update}\ f_m(x) = f_{m-1}(x) + \alpha \sum_{j=1}^{J_m} \gamma_{jm} I(x \in R_{jm})$$

What is $f(x)$? $\hat{y}$. And $\hat{y}$ is the predicted Output column (predicted_salary) quantified by a ~~Tree~~ model. α = learning-rate.

Since $m=1$,

$$f_1(x) = f_0(x) + \alpha \cdot \sum_{j=1}^{J_1} \gamma_{j1} I(x \in R_{j1})$$

$f_1(x)$: The can_predict value on the actual Output column of the first Tree model.

$f_0(x)$: Predicted value (mean of y) of ~~the~~ Base model.

$\sum_{j=1}^{J_1} \gamma_{j1} I(x \in R_{j1})$: The predicted value (residual error) of the 1st Tree model.

$\Rightarrow f_1(x) = f_0(x)$ + α·1st_Tree Model's prediction.

$\Rightarrow$ predict_y_1 = predicted_y_0 + α·predicted_residual_1

if $m = 2$,

$$f_2(x) = f_1(x) + \alpha \cdot \text{predicted_residual_2}$$

$$\therefore f_2(x) = f_0(x) + \alpha \cdot \text{pred-res-1} + \alpha \cdot \text{pred-res-2}$$

For each Tree model,

$$\text{pred_y}, f_m(x) = f_0(x) + \sum_{i=1}^{m} (\alpha \cdot \text{pred-res-i})$$

Step 3: Output $\hat{f}(x) = f_M(x)$

$\hat{y} = \hat{f}(x)$. This $\hat{f}(x)$ denoting the official output column on the actual output column of the Gradient Boosting Regressor after all the tree models trained i.e. after step 2.

This value is actually the last tree model's pred_y, $f_m(x)$

$$= f_0(x) + \sum_{i=1}^{m} (\alpha \cdot \text{pred_res_i})$$

because the last tree model is the combination of all the models.

Note:

1) In Step 2.C, we calculated the Leaf Node | Terminal Region's value manually and they were equal to the ~~plot_tree()'s "value" because~~ leaf nodes value of the tree we trained (see plot_tree() graph) because DecisionTreeRegressor(criterion="squared_error") use squared_error (by default) and we also used the same here as loss function.

If we used different loss function, our manual leaf node | terminal region's value wouldn't be same with DecisionTreeRegressor(...).

Scanned with CamScanner

—> Summary (simplified for loss = squared error)

---

1. Quantify Base Model's prediction on output column, mean and the residual error.

2. For each DecisionTreeRegressor (DTR) model :
    a. Fit the DTR with Input Columns X and output column Residual_Error_of_previous_model.
    b. Quantify the **residual error** this current DTR **predicted** from its training data.
    c. Quantify the **residual error (Actual_output_column - Predicted_output_column)** this current DTR made.
        1. **Predicted_output_column** = Base_model's_prediction + Σ(α * DTR's predicted_residual_error)
        [This Σ is from first DTR model to current DTR model]

3. The last Tree model's **Predicted_output_column** is the official output of the Gradient Boosting Regressor after m(number of models) boosting iterations i.e. after step 2, on the training dataset(x_train, y_train).

This is the same steps I explained in "Gradient Boosting Regression Theory.pdf".

Testing GradientBoostingRegressor

I've already discussed it in "Gradient Boosting Regression Theory.pdf", under the "Gradient Boosting on Testing Dataset" section. There is no math behind it. I'm just going to copy paste it below :

We've seen how Gradient Boosting Regressor predicts the output column, salary, on training dataset. But how does it predict for new data(iq, cg)? E.g., for cg = 3.9, iq = 10 what is the Predicted salary($)?

Wait! How did we predict the training dataset for 2 tree models?

pred_salary2 = pred_salary0 + (learning_rate * pred_res1) + (learning_rate * pred_res2)

pred_salary0 = The Base model is actually No Model which returns the mean of the output column which is just a scalar value.
pred_res1 = We train the first tree model on (iq, cg, res0). Then we predict each training input (iq, cg) and for each training input we get residual_error.

pred_res2 = We train the second tree model on (iq, cg, res1). Then we predict each training input (iq, cg) and for each training input we get residual_error.

So each model is trained on the same input data and it returns|predicts Residual Error which we use to quantify Predicted Salary for that model.

Now, after training the GradientBoostingRegressor, to predict the salary for cg = 3.9, iq = 10, we send this data to each tree model (the base model's predicted_value is constant since its not a tree model) and each tree model returns the Residual Error as its prediction which is assume pred_res1 = 9.7, pred_res2 = -8 and pred_salary0 is 42 from the training.

So predicted_salary = pred_salary0 + (learning_rate * pred_res1) + (learning_rate * pred_res2)

= 42 + (0.1 * 9.7) + (0.1 * -8)

= 42.17 $

This is how prediction happens on unseen|testing data in Boosting. For each testing data : We give to each Machine Learning Model and each ML Model returns something(Residual Error or predicted_salary etc) from their prediction i.e. model.predict() and we add all the “something” in a specific way to get the final Predicted Output.